

LazyTensor: combining eager execution with domain-specific compilers

Alex Şuhan

Facebook*

Davide Libenzi

Google Research, Brain

Ailing Zhang

Facebook

Parker Schuh

Google Research, Brain

Brennan Saeta[†]

Google Research, Brain

Jie Young Sohn

Google, Cloud AI

Denys Shabalín

Google Research, Brain

Abstract

Domain-specific optimizing compilers have demonstrated significant performance and portability benefits, but require programs to be represented in their specialized IRs. Existing frontends to these compilers suffer from the “language subset problem” where some host language features are unsupported in the subset of the user’s program that interacts with the domain-specific compiler. By contrast, define-by-run ML frameworks—colloquially called “eager” mode—are popular due to their ease of use and expressivity, where the full power of the host programming language can be used. LazyTensor is a technique to target domain specific compilers without sacrificing define-by-run ergonomics. Initially developed to support PyTorch on Cloud TPUs, the technique, along with a substantially shared implementation, has been used by Swift for TensorFlow across CPUs, GPUs, and TPUs, demonstrating the generality of the approach across (1) **Tensor** implementations, (2) hardware accelerators, and (3) programming languages.

1 Introduction

Imperative, sequential program execution—colloquially called “eager execution” or “define-by-run” [32] in ML contexts—is easily understood and expressive, which is why it is used as the basis for most widely-adopted programming languages. Popular libraries for machine learning centered around eager execution such as PyTorch [26] and NumPy [10] are known to be both flexible and easy to debug. Programs using these libraries dispatch “kernels”—pre-compiled functions such as matrix multiplication, convolution, or element-wise arithmetic operations on **Tensors** (n -dimensional arrays)—to computational devices (e.g. CPUs or GPUs).

On the other hand, optimizing domain-specific compilers (DSCs) [18, 4, 29] substantially improve performance of machine learning models. Additionally, these compilers are sometimes the *only* way to target domain-specific accelerators, such as Cloud TPUs [15]. The downside: a user’s program must be presented to these DSCs in a compiler-specific intermediate representation (IR). Because these IRs are focused on a particular domain, they typically do not aim to be as expressive as general-purpose programming languages. While numerous libraries in general-purpose programming languages have been developed to build these IRs, they all suffer from the *language subset problem* where expressivity is sacrificed in the portion of the user’s program that uses the library to align with the capabilities of the target IR.

In this paper we introduce *LazyTensor*¹, a novel approach to combine the benefits of eager execution with DSCs. Our technique allows full use of all host programming language features throughout the **Tensor** portion of the user’s program, avoiding the language subset problem. Initially

*Work done at Google Research, Brain

[†]Correspondence to saeta@google.com

¹Unrelated to **LazyTensor** in <https://pytorch.ai/>

developed to support PyTorch on Cloud TPUs, the LazyTensor approach has been adopted by two numerical computing libraries in two different programming languages while sharing the majority of the implementation. The main contributions of this paper are:

1. A technique for combining an eager programming model of Tensor programs with domain specific compilers that does not restrict the expressivity of the user’s programming language. The approach is general enough to be applied to any define-by-run machine learning framework. (Section 3)
2. An implementation of the LazyTensor design across two different machine learning frameworks in two different programming languages: PyTorch and Swift for TensorFlow. (Section 4)
3. An evaluation of our design across multiple languages, Tensor types, and accelerators (GPUs and TPUs). (Section 5)

2 Background

Deep learning models are often trained using libraries centered around a multi-dimensional array abstraction, often called a **Tensor** [10, 26, 1]. The model is (1) a collection of **Tensors** corresponding to learned parameters (weights) and (2) a sequence of operations mapping (a) the parameter collection and (b) input **Tensor**(s) (e.g. a batch of images represented as a 4-dimensional array²) to the output **Tensor** result (e.g. a 2-dimensional³ **Tensor** corresponding to a one-hot encoding classification category per image).

Domain-specific optimizing compilers have been developed around the **Tensor** abstraction to (a) target domain-specific hardware such as TPUs, and/or (b) eke the maximum performance out of a given hardware footprint. These domain specific compilers consume source programs as input in compiler-specific intermediate representations (IRs), e.g. XLA HLO IR [18]. These IRs are not designed to be human authored, as they either have extremely verbose syntax, or inflexibility (e.g. no dynamic memory allocation, and require static-shapes for every **Tensor**).

2.1 Graph Mode

Historically, many deep learning frameworks [1, 14, 31] represent models as data structures. The TensorFlow v1 [1] system is organized around the construction and execution of dataflow graphs. A TensorFlow Python program acts a meta-program, where the Python code builds a computational graph. This graph is handed off to the dataflow execution engine, implemented in C++. Because this graph can be serialized as a GraphDef protocol buffer, and because it executes independently of the host programming language, we refer to it as the GraphDef programming language, and the dataflow execution engine as the interpreter of the GraphDef programming language. Because the GraphDef programming language is optimized for **Tensor**’s and does not support many of Python’s features,⁴ it can be translated into an equivalent program in a DSC’s IR.

2.2 Eager programming & asynchronous execution

In contrast to graph mode, define-by-run libraries [32, 26]—where the neural network model is written in code and directly executed—are seen as easier to debug and more flexible, because users have the full power and expressivity of a general purpose programming language. Compute hardware (CPUs as well as accelerators like GPUs) are efficiently utilized through asynchronous execution. When a user’s program requests a matrix multiplication between two **Tensors**, the operation’s “kernel” is dispatched

²The **Tensor** can be arranged as [batch × image height × image width × image channels].

³The shape corresponds to: [batch × #-classes].

⁴Including exceptions, reflection, dynamic dispatch, and more.

to the compute device, and control is immediately returned to the user’s program. The runtime blocks the user’s program only when it attempts to view the contents of a `Tensor`. One can think of the `Tensor` type as a *promise* to return a concrete `Tensor` value at some future time. Because the `Tensor` computation is never materialized in a data structure, there is not a program representation that can be translated to a DSC’s IR.

2.3 Eager programming & DSCs

There are a number of mechanisms for staging code out of an eager `Tensor` programming environment so that it can be optimized by a DSC.

Tracing. One set of methods involves a user-directed mechanism for running eager program code, with specialized `Tensor` types or in a specialized context, while recording information about what `Tensor` operations were executed. This tracing process is akin to that used by operator-overloading reverse-mode automatic differentiation systems like Autograd [20] to build a “tape” that is walked backwards during gradient evaluation. Systems like JAX⁵ [9] and TensorFlow 2 [2] provide tracing decorators that cause annotated functions to be executed in a context that abstracts `Tensors` into *representations* of potential values and captures `Tensor` operations for optimization by a DSC. On its own, tracing with abstract values is essentially a more-ergonomic interface for a “graph mode”, while host language features like control flow and all non-`Tensor` code are “invisible” to tracing and either are executed only at trace time or, if dependent on runtime `Tensor` values, cannot be traced at all.

Language virtualization. Tracing can be augmented by a source-to-source transformation that rewrites untraceable language features like control flow into traceable ones. This “virtualization” process is exemplified by the Lightweight Modular Staging (LMS) [28] framework for Scala and by AutoGraph [23], a Python virtualization system adopted by TensorFlow 2. Once augmented with virtualization, tracing systems are able to stage out eager code even if it uses operations on built-in types like Python lists whose behavior can’t be overloaded, or language-native control flow that branches on runtime `Tensor` values. Virtualization may not cover all language features, with exception-based control flow forming an especially difficult case.

Direct compilation. Another approach to bridging eager code and a DSC is to implement a compiler frontend. TorchScript implements a lexer, parser, type inference engine, and optimizer for a `Tensor`-focused subset of Python that includes most real-world PyTorch code, while Julia’s XLA support [8] leverages the Julia compiler to do the same for a subset of that language. When embedded in dynamic languages like Python and Julia, such a compiler can be invoked just-in-time, with a user interface similar to tracing (e.g. a function decorator). Like tracing, approaches based on a compiler frontend typically require that all code in the staged-out function either be statically evaluated⁶ at compile time or be present in the final program handed off to the DSC. This restriction is viral: compilation of one user function requires compilation of functions that it calls, so unsupported behavior in a function or library (e.g., a call to a foreign function like a physics simulator or environment model) means that all transitive callers also cannot be compiled.

A key common downside to these existing techniques is that they only support a subset of the host programming language. Additionally, these approaches prevent the user from interleaving code that should be compiled with a DSC and code that shouldn’t. This is similar to the “function coloring problem” from `async-await` contexts [25]; functions not compiled with a DSC can call functions compiled with a DSC, but not the other way around.

⁵An acronym for, among other things, “JAX is Autograd and XLA”.

⁶If this static evaluation is not performed, or performed using an alternative language implementation (as in TorchScript), some language features may not be supported at all.

3 The LazyTensor Approach

Types in common general-purpose programming languages operate with eager semantics; because our approach maintains the *illusion of eager execution*, our approach does not compromise the host programming language. In particular we support the complete host language, including interleaving arbitrary `Tensor` and non-`Tensor` computations.

Our approach build upon the insight behind PyTorch’s asynchronous execution: as long as the user program does not observe a `Tensor`’s contents, *the user’s program cannot distinguish when a `Tensor` operation is actually executed.*⁷ Instead of dispatching each op individually to execute asynchronously, we *buffer sequences of operations in a data structure*. Additionally, we observe that these operation sequences can be *transformed into a DSC’s IR*. The programs generated by a DSC are semantically equivalent to running the operations individually, and thus we can *replace executing the operation sequence with direct invocation of the compiled DSC program*. *Tensors are neither promises nor representations of future data, but both simultaneously!*

We call these “sequences of operations” an *IR graph*. The IR graph construction process always begins from a `Tensor` operation. This reflects the separation between the host language and the LazyTensor library: calls to `Tensor` operations are the only entry points to our system. The first such `Tensor` operation is always a “factory method”, which creates a `Tensor` from an existing `Tensor`, a range, a repeated constant value, or random values sampled from a specified distribution.

The entire `Tensor` API can be divided into two domains, operations that can be represented in a IR graph (*IR compatible operations*), and operations that force the evaluation of a IR graph (*IR incompatible operations*). Any `Tensor` operation that exclusively returns one or more `Tensor`’s fall into the first domain. Examples include matrix multiplication, convolution, and element-wise arithmetic. Operations on `Tensors` that return non-`Tensor` types are incompatible with IR construction, such as operations that return a scalar value (e.g computing a string-representation to print to a console, or a boolean to make data-dependent control-flow decisions in the host programming language).

The entire `Tensor` API is available at all points in the user’s program. This has a number of advantages including the ability to use the complete host language for:

1. **Function abstraction.** IR graph recording happens transparently through host-language abstractions such as functions. We need not differentiate functions which are incompatible with the IR graph, and thus avoid the function coloring problem. Any function can call any other function, irrespective of whether it is composed exclusively of IR compatible operations. *Functions that call IR-incompatible operations simply force the evaluation of the IR graph at runtime, and a new IR graph is subsequently started. Changes to the implementation of one function never affect the ability to optimize any other function.*
2. **Control flow.** Because we maintain an eager API, the complete set of host language control flow mechanisms function identically to an eager implementation. This includes all host language control-flow operations including complicated cases such as exceptions or virtual function calls.
3. **Data structures.** Programs can embed `Tensor` values as part of arbitrary data structures. This allows for composition with non-`Tensor`-aware libraries which could embed `Tensor` values as boxed (in dynamic languages) or type-specialized first-class values (in compiled languages).

As a result, in contrast to prior staging systems, we make no compromises on the integration with the host language. We achieve this by effectively building a sophisticated version of the *eager runtime that relies on recording an IR graph behind-the-scenes, rather than exposing it to the user.*

⁷Ignoring timing and similar side-channels.

4 The LazyTensor System

The LazyTensor system builds on top of (a) an underlying eager runtime (either PyTorch or TensorFlow-Core) and (b) the XLA domain-specific compiler. LazyTensor has 3 main components: (1) a custom `Tensor` type with an identical API to an existing `Tensor` type, (2) a mapping from the high-level `Tensor` operations to XLA HLO sequences implementing the semantics of the requested operation (called a “lowering”), and (3) a runtime that lowers sequences of `Tensor` operations into XLA HLO IR, and orchestrates compilation & execution of the resulting program. Because compilation often is very expensive, the LazyTensor system carefully **cached and re-uses program IR graphs** keyed on a canonicalized form.

The LazyTensor implementation **includes an additional “barrier” API** (`mark_step()` in PyTorch, `LazyTensorBarrier()` in Swift for TensorFlow). This API completes the **current in-progress IR graph construction, and dispatches it to the runtime for compilation and execution**. The barrier API takes a boolean parameter to control whether the call should block until IR graph execution has completed and all `Tensor` data has been materialized in memory. Implementations of IR incompatible operations call the barrier API with the blocking bit set before proceeding with their implementation. Future work can remove the barrier API from the public interface.

The barrier needs information regarding all live `Tensors` for which the outstanding computation accumulated for the step needs to be executed to set their value. The liveness information is tracked per device by a context object, `DeviceContextArena`, available for the entire lifetime of the process. Constructors and destructors on the `Tensor` type call `RegisterTensor` and `UnregisterTensor` methods (respectively), using unique identifiers generated when tensors are created.

The core of the LazyTensor system is implemented in C++, and has been substantially shared across both PyTorch and Swift for TensorFlow [30]. The custom `Tensor` types are in Python and Swift (respectively), and are thus not shared. Further, because the `Tensor` APIs are not identical, the mappings from user-level operations to XLA HLO are tweaked accordingly.

The LazyTensor system has a number of other features to make the system useful in practice. LazyTensor supports **distributed training**, including leveraging the custom high-speed chip-to-chip network on TPU Pods by exposing the relevant XLA HLO collective operations (e.g. cross-replica-sum). Finally, LazyTensor adds support for **automatic mixed precision** [22] enabled by an environment variable.

4.1 IR graph representation

The LazyTensor IR graph **records a computation as a directed acyclic graph**, in which the leaves are the inputs and the roots are the results computed based on the given inputs. Figure 1 contains two representations of the IR graph constructed from PyTorch for a simple $x * y + z$ computation, on floating point tensors of shape $[2, 4]$.

All the nodes record their shape in addition to the operation type itself, in this case `f32[2,4]` for the shapes, with the exception of the scalar constant 1 which is represented as rank zero: `f32[]`. Inputs are represented as `xla::device_data`.

The multiplication and addition are represented by nodes `%6` and `%7`. In PyTorch, the addition operation allows specifying a scaling factor for the second parameter, which is represented by the additional node `%3`. The scaling factor is the constant 1 expanded to the required shape. The scaling operation is optimized away by the underlying XLA compiler backend. The generated native code will not materialize the constant nor execute the multiplication by 1.

Optimizing away operations which involve known scalar values must be balanced against the ability to reuse previously-compiled code for IR graphs which differ only in such values. To achieve the latter, **scalar values could instead be treated as computation parameters**. For example, certain classes of models involves indexing at a variable starting position and a fixed length—compiling a separate version for each starting index doesn’t improve performance on any class of hardware and the user experience is degraded by the increased compilation times. We’ve chosen a simple heuristic:

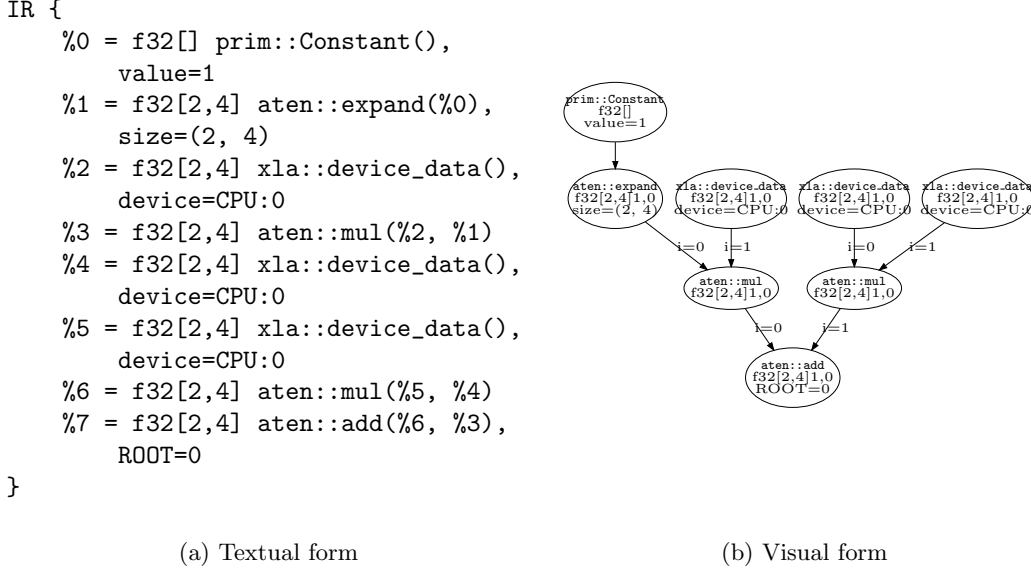


Figure 1: An IR graph of $x * y + z$.

treat 0 and 1 as special scalars, which have their values encoded in the IR graph while treating the rest as dynamic computation parameters. This covers the elision of multiplication by 1 or adding zero-initialized gradients that result from lowering some high level APIs. While this might trigger spurious recompilations due to incidental occurrences of special constants, we haven’t observed this to be a problem in practice.

Analogously to native PyTorch, there are no implicit transfers between devices. This is reflected in the IR graph representation above: inputs, of type `xla::device_data`, are associated with a device (which in this case is `CPU:0`). All operations require all inputs are on the same device address space and subsequently their outputs stay on the same device. While at first glance this choice appears to place a big burden on the user, frameworks offer ways to transfer entire models to a device with one or two lines of code. In addition, this choice prevents difficult to debug performance problems associated with implicit transfers between device address spaces which the user didn’t request.

4.2 Control Flow

Currently, there is no representation for control flow in LazyTensor IR graphs. For both conditionals and loops, the resulting execution path is captured and executed. Given a simple program with a loop (Figure 2a), an unrolled, linear IR graph is generated (see Figure 2b).

This reflects the separation between “tensor programs” and the host language (Python or Swift in our case). While extending the system with control flow would be possible, generating IR graphs with control flow would require a degree of integration with the host language. Moreover, our implementation works well in practice. For example, conditional statements that select between a small number of model configurations or a loop that constructs a model out of repeated layers are both well served by this choice. On the other hand, dynamic conditional statements that branch on runtime Tensor values simply cause a break in the trace, rather than making tracing impossible.

4.3 In-place operations

Both Swift for TensorFlow and PyTorch allow syntactically in-place updates: `s += x` updates the value of `s` to `s + x`. Such operations are the cornerstone of weight updates during training and must

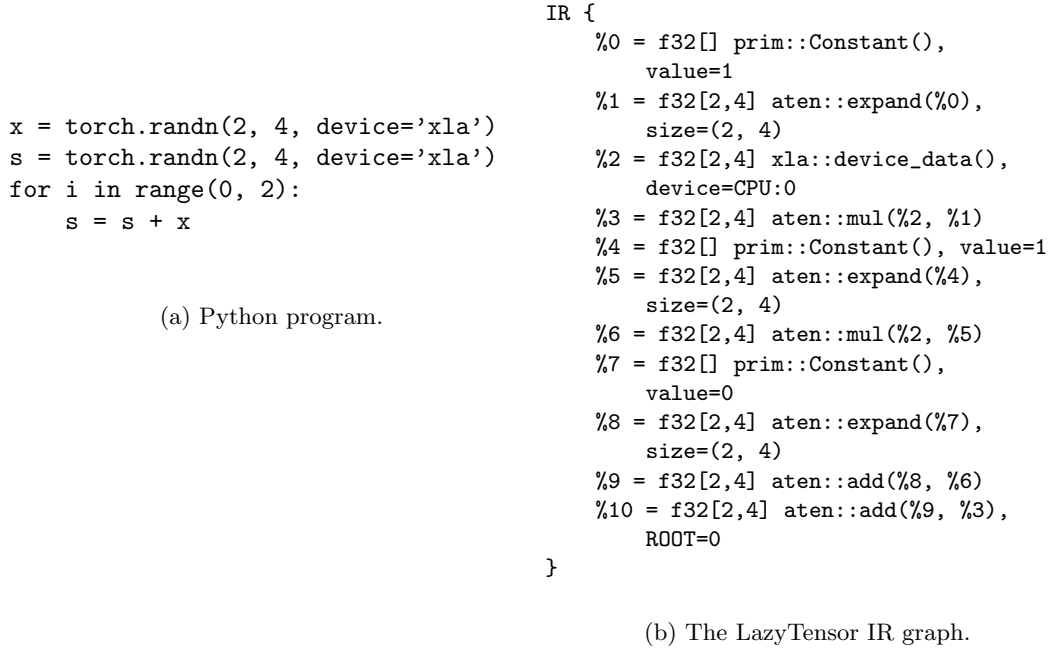


Figure 2: A simple program with a loop.

be supported efficiently.

The two frameworks diverge in the way they support such operations. Swift’s operator overloads automatically convert the in-place version to the simple assignment `s = s + x` and therefore can reuse the regular addition implementation. The XLA compiler will leverage the knowledge that the old value of `s` is no longer needed and can reuse memory efficiently.

On the other hand, Python doesn’t offer such a rewrite and therefore PyTorch requires the implementation of additional, in-place versions of arithmetic (among other) operators. However, the LazyTensor IR graphs have no concept of mutation. Instead, all IR graphs represent pure functions. This was chosen to accurately reflect the underlying XLA HLO IR.

To achieve mutation semantics, the LazyTensor system implements mutation as a substitution of the underlying computation associated with the destination with the computation associated with the expression on the right hand side of an in-place operation. This achieves the same net effect as the built-in mechanism in Swift: the generated IR graph looks the same for regular and in-place operations. We rely on runtime indirection to replace the underlying computation to achieve the desired semantics.

While mutation semantics can be achieved in a purely functional manner, memory usage and performance are crucial when training machine learning models. A naive implementation of this model would use twice the optimal amount of memory for updates, due to the need to store the right hand side before replacing the destination with it. Fortunately, the purely functional representation we’ve chosen matches the model of XLA, which allows specifying aliasing between input and output buffers⁸. Destinations of in-place operations that end up as parameters of a LazyTensor IR graph can be “donated” for reuse as outputs through this mechanism, solving the problem of memory usage.

However, use of aliasing must be limited to IR graphs terminated at the step barrier. Consider the following pseudo-code sequence:

```

# Assume Tensor a is materialized data
a += 1

```

If we use aliasing for the IR graph rooted at `a`, the following sequence would behave incorrectly:

⁸Details at <https://www.tensorflow.org/xla/aliasing>.

```
print(a)
print(a)
```

With aliasing enabled, both `print(a)` statements would increment the buffer `a`, unless we force the evaluation of updated value of `a` instead of growing the IR graph.

Additionally, we must make sure that all live tensors are part of the computation when we use aliasing. Consider the following sequence:

```
# tensor a is immediate data
b = a + 2
a += 1
print(a)
print(b)
```

If `b` is not part of the same IR graph as `a`, it'll use the updated value of `a`, which is incorrect.

Because of these two caveats, we limit our use of aliasing to computations which meet the following two conditions:

- The computation behind each output is evaluated and not allowed to grow any further.
- All live `Tensors` are part of the computation.

Both conditions are met at the end of the training step when `mark_step()` is called inside `optimizer.step()` in PyTorch, as we accumulate both the complete forward and backwards pass. In practice, this technique is only necessary to minimize memory usage for gradient updates, which also happen at the end of the training step.

4.4 In-place operations on views

PyTorch offers views as an explicit mechanism to control the sharing of underlying storage between `Tensors`. Several operations in its API return results which are guaranteed to share their underlying storage with their base `Tensors`. For example, in the following snippet of code:

```
t = torch.rand(4, 4)
v = t.view(2, 8)
```

`v` is a reshape to size `(2, 8)` guaranteed to share its storage with `t`. There are several other operations besides `view` itself which guarantee sharing semantics, some of which only operate on a subset of the storage (e.g. `narrow`, `permute`, etc).

In a purely functional paradigm, this feature has no implications as long as user programs respects referential transparency. However, PyTorch allows in-place updates on views, which are guaranteed to be reflected in the base `Tensor` as well. For example, `v.add_(1)` will also update `t` since `v` is a view of `t`.

Our system supports this feature correctly, extending the approach used for in-place operations on regular tensors. An update of a view creates two sequences of operations (still in the same IR graph):

1. The “forward” sequence, which creates the computation required to represent the updated view. Multiple view operations can be applied to the base `Tensor` and we need to iterate through all.
2. The “backward” sequence, which creates the computation required to represent the updated base. We start from the updated value of the view, iterate the view operations in reverse order and apply the inverse of each view operation.

```

IR {
  %0 = s64[] prim::Constant(), value=1
  %1 = s64[] xla::device_data(), device=CPU:0
  %2 = s64[] aten::mul(%1, %0)
  %3 = f32[2,3,4] xla::device_data(), device=CPU:0
  %4 = f32[3,4,2] aten::permute(%3), dims=(1, 2, 0)
  %5 = f32[3,4,2] aten::add(%4, %2)
  %6 = f32[2,3,4] aten::permute(%5), dims=(2, 0, 1), ROOT=0
}

```

Figure 3: A IR graph involving in-place operations.

For example, if x is a tensor of shape (2, 3, 4), $v = x.\text{permute}(1, 2, 0).\text{add_}(42)$ leads to the IR graph in Figure 3.

The resulting IR graph contains both the permutation directly specified by the user—in this case (1, 2, 0) and its inverse, (2, 0, 1). The former is used to compute the underlying computation which represents v , while the latter is used for representing the updated value of x , the source of the view.

4.5 Unimplemented Tensor Operations

Machine learning frameworks offer thousands of operations in their **Tensor** APIs. Although DSCs often support more flexible linear algebra operations than libraries of pre-compiled kernels, **DSCs often do not support all Tensor operations**. Examples include image decompression algorithms. In order to deliver a *drop-in* replacement **Tensor** API, all **Tensor** operations that do not have lowerings to the DSC are implemented with the following pattern:

1. Evaluate the IR graph for all operation inputs.
2. Call the underlying eager implementation on the evaluated inputs.
3. Start a new IR graph with the eager op execution’s result.

Thus, every **Tensor** operation available on the type will result in a correct program when executed with the LazyTensor system, albeit with some potential performance implications. Using a debugger’s breakpoint facility, users can quickly determine where their code calls operations with no implemented DSC lowering.

5 Evaluation

We evaluate the LazyTensor system across a number of dimensions and applications.

5.1 Code Reuse

The core of the LazyTensor system has been used to power XLA integrations across both PyTorch and Swift for TensorFlow. Table 1 documents the source lines of code (SLoC) within the respective folders of the implementations in Swift for Tensorflow, and annotates whether they are shared between the two frontends. Because this measure is somewhat crude, we estimate slightly above 75% of the SLoC are shared, despite the folder-based method of estimation indicating around 85% SLoC reuse. This demonstrates this technique—and a substantial fraction of the implementation—is reusable across programming languages, and underlying eager runtimes.

Component Folder	SLoC	Shared?
<code>xla_tensor</code> & <code>xla_client</code>	49,910	Yes
<code>swift_bindings</code>	7,325	Swift-only
<code>CX10</code>	1,697	Swift-only

Table 1: Source lines of code (SLoC) in Swift for TensorFlow used to implement individual components of the system as grouped by folder, and whether they are shared between the Swift and Python implementations of the LazyTensor technique.

Framework	Accelerator	Global Batch Size	Eval Perplexity	Training Time
PyTorch Native	4x V100	48	3.14	133.4 minutes
PyTorch LazyTensor	Cloud TPU v3-8 (4x TPUv3 chips)	128	3.25	38.3 minutes

Table 2: Time required to train HuggingFace’s **roberta-base** (125M params) on the raw WikiText-103 dataset [21] for 3 epochs using half precision. The largest batch size that was able to fit in each chip’s memory was used for above numbers to maximize utilization and thus optimize training time.

5.2 Training Transformers on Cloud TPUs

The Transformer [33] is a deep learning architecture widely used in the natural language processing domain today, which has resulted in state of the art performance on various metrics everywhere from language parsing, machine translation, to question answering. This architecture has heralded a wide lineage of attention based models such as BERT [7], T5 [27], and more.

With the PyTorch LazyTensor implementation, we’ve enabled the popular HuggingFace Transformer library [34] to run on Cloud TPUs using XLA. PyTorch with LazyTensor was able to demonstrate significant performance improvements on TPUs compared to roughly equivalent GPU hardware (Table 2).

5.3 Scaling ResNet-50 on TPUs

We evaluate the scaling properties of the LazyTensor system by using Swift for TensorFlow to train ResNet-50 [12] on TPU Pods. The performance of Swift for TensorFlow on TPUs was measured by training the ResNet-50 image classification network [12] on the ImageNet 2012 dataset [6] using TPUv3-16, TPUv3-32 and TPUv3-128 clusters (not using Cloud), shown in Table 3. The model was trained for 90 epochs, and both the time required as well as the post-warmup throughput in examples per second were recorded. Per-accelerator throughput is substantially maintained, demonstrating that the LazyTensor technique can scale to large TPU super-computers.

# Cores	Validation Accuracy (top-1)	Training Time (90 epochs)	Throughput (examples / s)	Per-Accelerator Throughput (examples / s / TPU core)
16	78.1%	189 minutes	10164	635.25
32	77.7%	96 minutes	20015	625.47
128	77.8%	25 minutes	77726	607.23

Table 3: Swift for TensorFlow training performance for ResNet-50 on ImageNet on TPUv3 clusters. Per-accelerator throughput is largely maintained while scaling from a single host to 8 hosts synchronously training a single model in data-parallel fashion, demonstrating the scalability of the LazyTensor approach employed by Swift for TensorFlow to target TPUs.

Operation & Length	Eager (ms)	LazyTensor (ms)	Ratio
Score 4	96	65	0.68
Score 8	182	118	0.65
Score 14	282	165	0.59
Score & Gradient 4	425	281	0.66
Score & Gradient 8	894	581	0.65
Score & Gradient 14	1350	853	0.63

Table 4: Time spent in respective operations of the WordSeg algorithm [16] based on the different Swift for TensorFlow `Tensor` approaches when run on a GPU.

5.4 Limitations of XLA compilation

Unfortunately, not all models achieve higher performance using the LazyTensor system as compared to the eager-system equivalent. If programs do not run for long enough, the advantages of specialization do not outweigh the overheads of the JIT-compilation itself. As a result, this approach often only makes sense during long-running, iterated computations such as neural network training, or batch inference.

One of the most painful limitations comes not from the technique itself, but the underlying DSC: **XLA’s static shape limitation**. All `Tensor` shapes must be known at IR graph compile time, as they are used for static memory planning and other optimizations within the XLA compiler. Although the system caches XLA programs based on a canonicalization of the LazyTensor IR graph, some ML models never converge to a “shape stable” set of IR graphs. For example, training MaskRCNN [11] on the COCO [19] dataset displays poor accelerator utilization, as **the accelerator remains idle while XLA repeatedly compiles new shape-specialized accelerator programs**. This application validated our choice to faithfully implement an *identical API* to the original eager execution, as it enables users to pick whichever execution strategy works most effectively for their applications at any given time.

6 Related work

JAX [3, 9] also builds on top of XLA, but uses explicit tracing—versus LazyTensor’s implicit traces—to build optimizable program representations in their `jaxpr` IR. This tracing approach forces users to refactor their code if any single function in the user’s program calls some un-traceable functionality such as a black-box environment or uses non-`Tensor`-based data structures in a data-dependent way. Although DyNet [24] builds traces, and performs optimizations including automatic batching on the trace IR, it still dispatches ops in the trace eagerly via cuDNN [5] on GPUs and Eigen/MKL on CPUs instead of using a domain specific JIT compiler.

TensorFlow 1.X [1] users build an explicit graph data structure (`GraphDef`) using Python-based metaprogramming. The extra indirection inhibits debugging, and makes combining `Tensor`-based and non-`Tensor`-based algorithms more difficult. Although TensorFlow does have support for “partial run” to allow resuming the computational graph, this feature is incompatible with TensorFlow’s XLA support. TensorFlow 2 [2] avoids the explicit meta-programming of graph-building, but like JAX requires entire functions and all functions they transitively call to be traced and lifted into the IR. Taichi [13] and Numba [17] similarly JIT-compile subsets of Python. Our approach works across multiple languages, and allows for ergonomic mixing of code that cannot be optimized with the domain specific compiler.

Julia’s XLA support [8] translates Julia IR to XLA HLO without the need for tracing. Unfortunately, because XLA does not support dynamic memory allocation, the subset of the Julia program translated to XLA must equivalently not use dynamic memory allocation. It is this restriction that caused Swift for TensorFlow to abandon this architecture (similar techniques are there called graph program extraction) and instead adopt the LazyTensor approach. Fortunately, the Julia language has a number

of other meta-programming mechanisms that combine with Julia’s JIT-based specializing runtime to generate families of programs, effectively working around limitations induced by XLA’s static memory model.

7 Future work

Although the LazyTensor system has been effectively employed in multiple applications, there are a number of directions of improvement to broaden its applicability.

Despite LazyTensor graph construction overheads not affecting time-to-convergence for most training workloads, updating the IR graph can affect workloads operating on small models, small tensor sizes or low batch sizes, as is often the case for inference. While our system skips expensive recompilation when encountering the same IR graph for a second time, such workloads could benefit from mitigation of graph construction overheads as well.

One possible direction would address the representation of the IR graphs. Reduction of IR node size and usage of data structures and algorithms which minimize pointer chasing have been successful strategies in more traditional just-in-time compilers such as WebKit’s FTL JIT ⁹. We could borrow such techniques in our system to expand its appeal to workloads which would require a lower overhead in maintaining the IR graphs.

Another direction would provide users with a way to guarantee that the underlying computation of a `Tensor` is going to remain the same across steps. In doing so, our system would be allowed to skip the IR graph construction as well, after constructing it in the usual way during the first iteration.

Further, allowing users to encode control flow in the optimized program through special program annotations could increase performance in certain circumstances such as data-dependent branching.

Finally, techniques to automatically truncate, re-roll loops and asynchronously dispatch IR graph fragments could eliminate the need for `LazyTensorBarrier()` in user code and mitigate re-compilations due to variable upper bounds for loops. However, doing so efficiently could require some degree of cooperation with the host language implementation. For example, while recognizing an unrolled loop pattern is possible entirely in the LazyTensor system, the host language could instead provide hints about the presence of such a pattern.

8 Conclusion

In this paper we’ve introduced LazyTensor, a general technique to combine eager execution with domain specific compilers that does not restrict the expressivity of the user’s programming language.

We have successfully implemented this technique in two programming languages for two machine learning frameworks: PyTorch and Swift for TensorFlow. We managed to reuse the majority of the implementation while targeting completely different languages and `Tensor` APIs.

Our evaluation shows that we can efficiently target hardware that is only accessible via a DSC (XLA), which can result in significant performance improvements as measured by the HuggingFace Transformers library on Cloud TPUs. Additionally, we have shown how this approach can scale to large distributed accelerator clusters.

References

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. TensorFlow: A system for large-scale machine learning. In *12th {USENIX} symposium on operating systems design and implementation ({OSDI} 16)*, pages 265–283, 2016.

⁹<https://webkit.org/blog/5852/introducing-the-b3-jit-compiler/>

- [2] Akshay Agrawal, Akshay Naresh Modi, Alexandre Passos, Allen Lavoie, Ashish Agarwal, Asim Shankar, Igor Ganichev, Josh Levenberg, Mingsheng Hong, Rajat Monga, et al. TensorFlow Eager: A multi-stage, Python-embedded DSL for machine learning. *arXiv preprint arXiv:1903.01855*, 2019.
- [3] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, and Skye Wanderman-Milne. JAX: composable transformations of Python + NumPy programs, 2018. URL <http://github.com/google/jax>, page 18, 2020.
- [4] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Haichen Shen, Eddie Q. Yan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: end-to-end optimization stack for deep learning. *CoRR*, abs/1802.04799, 2018.
- [5] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. cudnn: Efficient primitives for deep learning. *arXiv preprint arXiv:1410.0759*, 2014.
- [6] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*, 2009.
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics.
- [8] Keno Fischer and Elliot Saba. Automatic full compilation of Julia programs and ml models to Cloud TPUs. *arXiv preprint arXiv:1810.09868*, 2018.
- [9] Roy Frostig, Matthew James Johnson, and Chris Leary. Compiling machine learning programs via high-level tracing. *Systems for Machine Learning*, 2018.
- [10] Charles R Harris, K Jarrod Millman, Stéfan J van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.
- [11] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross B. Girshick. Mask R-CNN. *CoRR*, abs/1703.06870, 2017.
- [12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In *European conference on computer vision*, pages 630–645. Springer, 2016.
- [13] Yuanming Hu. Taichi: An open-source computer graphics library. *arXiv preprint arXiv:1804.09293*, 2018.
- [14] Yangqing Jia, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross Girshick, Sergio Guadarrama, and Trevor Darrell. Caffe: Convolutional architecture for fast feature embedding. In *Proceedings of the 22nd ACM international conference on Multimedia*, pages 675–678, 2014.
- [15] Norman Jouppi, Doe Yoon, George Kurian, Sheng Li, Nishant Patil, James Laudon, Cliff Young, and David Patterson. A domain-specific supercomputer for training deep neural networks. *Communications of the ACM*, 63:67–78, 06 2020.
- [16] Kazuya Kawakami, Chris Dyer, and Phil Blunsom. Unsupervised word discovery with segmental neural language models. *CoRR*, abs/1811.09353, 2018.

- [17] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: A LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, LLVM '15, New York, NY, USA, 2015. Association for Computing Machinery.
- [18] Chris Leary and Todd Wang. Xla: Tensorflow, compiled. *TensorFlow Dev Summit*, 2017.
- [19] Tsung-Yi Lin, Michael Maire, Serge J. Belongie, Lubomir D. Bourdev, Ross B. Girshick, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: common objects in context. *CoRR*, abs/1405.0312, 2014.
- [20] Dougal Maclaurin. *Modeling, inference and optimization with composable differentiable procedures*. PhD thesis, Harvard University, 2016.
- [21] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *CoRR*, abs/1609.07843, 2016.
- [22] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory F. Diamos, Erich Elsen, David García, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. *CoRR*, abs/1710.03740, 2017.
- [23] Dan Moldovan, James M. Decker, Fei Wang, Andrew A. Johnson, Brian K. Lee, Zachary Nado, D. Sculley, Tiark Rompf, and Alexander B. Wiltschko. Autograph: Imperative-style coding with graph-based performance. *CoRR*, abs/1810.08061, 2018.
- [24] Graham Neubig, Chris Dyer, Yoav Goldberg, Austin Matthews, Waleed Ammar, Antonios Anastasopoulos, Miguel Ballesteros, David Chiang, Daniel Clothiaux, Trevor Cohn, et al. Dynet: The dynamic neural network toolkit. *arXiv preprint arXiv:1701.03980*, 2017.
- [25] Bob Nystrom. What color is your function?, 2015.
- [26] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems*, pages 8026–8037, 2019.
- [27] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *arXiv e-prints*, 2019.
- [28] Tiark Rompf and Martin Odersky. Lightweight modular staging: A pragmatic approach to runtime code generation and compiled dsls. *ACM SIGPLAN Notices*, 46, 10 2010.
- [29] Nadav Rotem, Jordan Fix, Saleem Abdulrasool, Summer Deng, Roman Dzhabarov, James Hegeman, Roman Levenstein, Bert Maher, Nadathur Satish, Jakob Olesen, Jongsoo Park, Artem Rakhov, and Misha Smelyanskiy. Glow: Graph lowering compiler techniques for neural networks. *CoRR*, abs/1805.00907, 2018.
- [30] Brennan Saeta, Denys Shabalin, Marc Rasi, Brad Larson, Xihui Wu, Parker Schuh, Michelle Casbon, Daniel Zheng, Saleem Abdulrasool, Aleksandr Efremov, Dave Abrahams, Chris Lattner, and Richard Wei. Swift for TensorFlow: A portable, flexible platform for deep learning. *MLSys*, 2021.
- [31] The Theano Development Team, Rami Al-Rfou, Guillaume Alain, Amjad Almahairi, Christof Angermueller, Dzmitry Bahdanau, Nicolas Ballas, Frédéric Bastien, Justin Bayer, Anatoly Belikov, et al. Theano: A Python framework for fast computation of mathematical expressions. *arXiv preprint arXiv:1605.02688*, 2016.

- [32] Seiya Tokui, Ryosuke Okuta, Takuya Akiba, Yusuke Niitani, Toru Ogawa, Shunta Saito, Shuji Suzuki, Kota Uenishi, Brian Vogel, and Hiroyuki Yamazaki Vincent. Chainer: A deep learning framework for accelerating the research cycle, 2019.
- [33] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems 30*, pages 5998–6008. Curran Associates, Inc., 2017.
- [34] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. HuggingFace’s Transformers: State-of-the-art natural language processing, 2020.